Allocating What Programs Touch: Non-Binding Per-Site Reservations from Measured Memory Footprints

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A program asks the allocator for the memory it might need, and the operating system and a cluster scheduler must honour every request, although the program often touches only part of it. We use MemPrint, a Pin-based footprint profiler, to measure the bytes each allocation site requests and the bytes the program touches in each block, and we pass these measurements to the compiler and the runtime as *non-binding reservations*. The system has three parts. First, a per-site mode of the Pin tool records each large block's touched bytes, highest touched offset and touched pages when the block is freed. Second, `MemPrintResize`, an LLVM 21 pass plugin, routes every allocation call through a runtime with a stable site identifier computed from the call's inlined debug location chain, passes `std::vector`'s growth capacity through the runtime, and can embed the reservations in the binary. Third, `libmpr`, a runtime that is either linked or preloaded, commits only a block's predicted prefix and maps the rest `MAP_NORESERVE`, so that an under-prediction costs commit accounting and never correctness. `libmpr` also skips the zeroing of pages that a program has never touched and keeps those pages out of transparent huge pages. Every program we ran produced its original output under `libmpr`. On darknet AlexNet inference, `libmpr` reduces peak commit by 46.8%. On miniVite, the pass's predicted vector capacity reduces peak RSS by 11.5% and peak commit by 22.5%. On llama.cpp, which zeroes a KV cache sized for an 8192-token context, `libmpr` reduces peak RSS by 49.1% and peak commit by 34.5% with a 135M-parameter model (43.3% and 25.1% with a 360M model), at no cost in run time, and reservations profiled on other inputs do as well. Across a 23-program HPC and interpreter corpus, where most programs already hold about what they touch, `libmpr` changes no output, keeps run time within -9% to $+6\%$, and reduces total peak commit by 10.2%. We also report what the measurements show about footprint estimation itself: a footprint curve drawn on wall-clock time needs 24% less headroom to cover the resident set than one drawn on memory references, and bytes touched can exceed the resident set when a program writes zeros to fresh memory.

1 Introduction

A program's memory request and its memory use are different quantities, and every layer below the program plans for the request. The allocator maps what is requested, the kernel charges the mapping to the commit limit [10], and a batch scheduler on a cluster with disaggregated memory reserves the request, or an estimate of it, before it places the job [19, 8]. Our earlier work built MemPrint, a Pin-based profiler [12, 15] that measures the bytes a program touches over its run, and showed that a scheduler which knows a job's footprint curve rather than its request can pack jobs much more tightly [14]. However, a scheduler that reserves less than a program requests does

not change what the program asks for. The program still allocates the full request, and the kernel still charges it.

The gap between request and use is large in some programs. darknet [17] running AlexNet inference commits 534.7 MB and touches 270 MB, because it allocates training-only weight-update buffers that inference never uses. llama.cpp [3] sizes its KV cache for the model's full 8192-token context and then writes zeros over all of it, so a 32-token completion holds 189 MB of cache in RAM and uses 0.4% of it. miniVite's edge list grows by doubling, so for a moment the old and new buffers are both live [4].

Therefore, we ask whether a measured footprint can be passed down to the allocator, so that the program obtains what it touches rather than what it requests, without trusting the measurement for correctness. Specifically, we treat MemPrint's per-site measurements as *non-binding reservations*. A block always receives every byte the program requested, but only the predicted part is committed, and the rest is mapped so that the kernel backs it only if the program touches it. A wrong prediction then costs accounting and never causes a fault the program can observe.

In this work, we make the following contributions:

- We extend MemPrint's Pin tool to profile requested against touched bytes per allocation site, with each block's touched bytes, highest touched offset and touched pages measured exactly when the block is freed (Section 4).
- We present `MemPrintResize`, an LLVM pass plugin that gives every allocation call a stable site identifier from its inlined debug location chain, routes it through a runtime, passes `std::vector`'s new capacity through the runtime, and embeds the reservations in the binary (Section 6).
- We present `libmpr`, a runtime that serves large blocks as a committed predicted prefix plus a `MAP_NORESERVE` remainder, grows blocks by moving page tables, and skips the zeroing of pages that were never touched (Section 7).
- We evaluate the system on darknet, miniVite, two LLM inference engines and a 23-program corpus, and we show that it halves llama.cpp's resident memory and nearly halves darknet's commit, with identical outputs and no time cost (Section 8).
- We report what the measurements show about footprint estimation: where miniVite's extra memory comes from, why the bytes a program touches can exceed its resident set, and why the time axis of a footprint curve matters more than its unit (Section 9).

2 Background

2.1 Three measures of a program's memory

We use three measures throughout, and the results depend on which one is meant.

Bytes touched. MemPrint's footprint is the number of distinct bytes the program has read or written in memory that is still live, measured from every memory reference under Pin [15]. With `-track_frees`, a freed range leaves the footprint. Bytes touched are a property of the program and its input, and they do not depend on the allocator or on the page size.

Resident set (RSS). The kernel counts the pages of a process that are in RAM. RSS includes file-backed pages, such as a memory-mapped model file and the code of every shared library, and it counts a transparent huge page as 2 MB [11]. We sample it from `/proc/<pid>/smaps_rollup` every

20 ms and report the peak sample. RSS is the quantity the kernel reclaims and the out-of-memory killer acts on, and it is the "RSS truth" of the scheduling study [14].

Commit. Linux charges private writable mappings to the commit limit when they are created [10]. We measure commit as the total size of the mappings that `/proc/<pid>/smaps` flags `ac` (accountable), sampled with RSS. A mapping created with `MAP_NORESERVE` is not charged. Under the default heuristic overcommit policy commit is only advisory, but under strict overcommit (`vm.overcommit_memory=2`) it is a hard limit, and in both cases it is the memory the system has promised to the process.

2.2 Why untouched memory is often free already

Linux backs anonymous memory lazily. A large `malloc` or `calloc` block is served by `mmap`, and a page of it becomes resident only when the program first writes it, or reads it, in which case the kernel maps a shared zero page that RSS does not count. Therefore, a block that the program never touches costs commit but no RSS. darknet's 250 MB of unused weight-update buffers are an example. Reducing RSS requires a change in how memory is obtained, not a smaller reservation. In our measurements RSS fell only where the program, its allocator, or the kernel touched pages the program did not need: vector growth that keeps two buffers live, a zeroing `memset` over fresh memory, and transparent huge pages that back 2 MB on the first touch.

2.3 Related work

Mesh compacts the heap of unmodified C and C++ programs by merging pages whose live objects do not overlap, and therefore reduces RSS after fragmentation, but unlike our system it does not use a profile of what the program will touch [16]. Maas et al. predict object lifetimes from allocation call stacks to place long-lived objects together [13], and HALO groups heap objects by profiled access affinity after link time [18]. Both use per-site profiles to change placement, whereas we use them to change how much of a block is committed and how it is backed. jemalloc and similar allocators return freed memory to the kernel more eagerly than glibc [2], but they still commit every requested byte. PagedAttention allocates an LLM server's KV cache in blocks as tokens arrive and therefore avoids reserving a full context per request [6]. Our llama.cpp result reaches a similar effect for an unmodified engine, without changing its code, because the untouched part of the cache is never backed. Zacarias et al. show that a 60% overestimate of memory demand reduces the throughput of a disaggregated system by about 25% [19], which is the scheduling motivation for both this work and our earlier study [14].

3 Overview

Figure 1 shows the pipeline. First, the program runs once under MemPrint's Pin tool with `-sites`, which writes one row per large block. Second, `sites.py` fits each site's reservation from one or more such runs. Third, the reservations reach `libmpr`. A program can use the system in two ways. In the *preloaded* configuration, `libmpr` is loaded with `LD_PRELOAD` into an unmodified binary, a site is the return address of the allocation call, and the reservations come from a file. In the *pass* configuration, the program is built with `MemPrintResize`, a site is a hash the compiler computes, and the reservations can be embedded in the binary. Only the pass configuration can change a vector's growth capacity, because the capacity is computed inside inlined library code, before any allocation call.

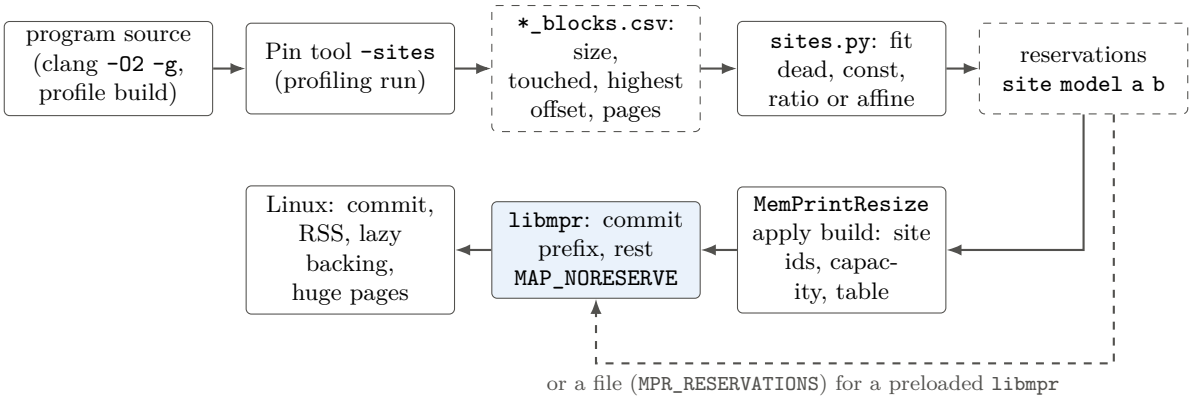


Figure 1: The pipeline. A profiling run of the program under MemPrint's Pin tool measures each large block; `sites.py` fits one reservation per allocation site; the reservations reach `libmpr` either embedded in a binary built with `MemPrintResize` or from a file when `libmpr` is preloaded into an unmodified binary.

4 Per-site profiling in MemPrint

The `-sites` mode of the Pin tool runs with the exact splitter, `-track_frees` and the bytes-touched footprint, which keeps one bit per byte for every touched page. It adds four mechanisms.

Site identity. The tool hooks the allocator's entry points (`malloc`, `calloc`, `realloc`, the aligned variants and every `operator new`). For the outermost allocator call of a thread it records the call's return address, written as the image name plus the address in the image file (for example `llama-completion+0x92266f`), which stays the same across runs despite address-space layout randomisation. Because `operator new` is hooked as an allocator, an allocation made by inlined standard-library code resolves to the program's own code. In a program built with `MemPrintResize`, the tool also hooks `__mpr_alloc` and the other runtime entry points, reads the site identifier from the argument the pass added, and names the site `id:0x...`.

Block summaries at release. When a release (a `free`, a `realloc` that moves, or an `munmap`) is applied, the tool is about to clear the block's bits from the footprint. Just before it does, it reads the block's touched bytes, the offset of its highest touched byte and its number of touched pages from the per-page bitmaps. Releases are queued in order with each thread's buffered references, so a summary sees every reference the program made to the block before freeing it. Blocks still live at exit are summarised at exit.

Allocator zeroing. `calloc` served from reused heap memory zeroes the block with a `memset` that is not the program's own use. The tool queues a probe at the `calloc` exit, in order with the references, and records the bytes touched at that point separately as `alloc_touched`.

Outputs. `*_blocks.csv` has one row per block of at least 64 KB with its site, kind, size, birth and death times, touched bytes, highest touched offset, touched and total pages, allocator-touched bytes, and whether it was grown from another block. `*_sites.csv` aggregates every block per site. The test suite checks exact values on programs written for the purpose: a dead block, a block touched only in a prefix, a block touched only at its tail (whose highest touched offset is its size), a `calloc` on a reused chunk, a `realloc` growth chain and a growing vector.

Two further options. In the course of this work we added `-clock seconds`, under which the timeline's snapshots are taken every N milliseconds of wall time and every row carries a `Seconds` column, and `-footprint pages`, which counts the 4 KB pages that hold a touched byte. We use both in Section 9.1.

5 Fitting reservations

`sites.py` reads the blocks of one or more profiling runs, keeps the blocks of at least 64 KB (the ones `libmpr` serves), and fits, per site, the number of bytes a block will need as a function of its requested size n. It fits the highest touched offset rather than the touched bytes, because `libmpr` commits a block from its start, so a prediction is safe for a block only if it reaches the block's last touched byte. Four models are considered:

- `dead`: 0, when no block of the site was touched;
- `const`: a, when the highest offset does not grow with the request;
- `ratio`: $a\,n$;
- `affine`: $a\,n + b$, the lowest line through two blocks that lies above the covered blocks.

A model *covers* a set of blocks if its prediction reaches each block's highest offset. Among the models that cover blocks holding at least a quantile q of the site's requested bytes ($q = 0.9$ by default), the fit keeps the one that commits the least over the training blocks. A site is left out when its best model would commit at least 95% of what was requested. The output is one line per site, `site model a b`, which `libmpr` reads at start-up.

A profile can come from the same input the program will run on (we call this *own profile*, which measures the mechanism) or from other inputs of the same program (*held out*, which measures how a reservation transfers). The quantile leaves some blocks uncovered by design, and Section 8.7 shows that under-predictions are safe.

6 The `MemPrintResize` compiler pass

`MemPrintResize` is a new-pass-manager plugin for LLVM 21 [7], 259 lines of C++, loaded into clang with `-fpass-plugin`. It registers the same module pass at two extension points of the optimisation pipeline, and it has three modes: `profile` (the default) routes allocations through the runtime, `apply` also embeds a reservation file, and `off` does nothing.

6.1 Allocation calls

The late instance runs at `OptimizerLastEP`, after inlining. For every call or invoke whose callee is one of the allocation functions in Table 1, it builds a call to the matching runtime entry point with the original arguments, a site identifier and a kind, replaces every use of the old call, takes over its name and erases it. `operator new` under an `invoke` becomes a plain call followed by a branch to the invoke's normal destination, because the runtime reports failure the way `malloc` does, and the landing pad loses that predecessor. Calls that `free` memory are not rewritten, because `libmpr` recognises its own blocks in `free`.

6.2 Stable site identifiers

A site identifier must name the same allocation in a profiling build and an apply build, and it must distinguish allocations that the optimiser has inlined into different callers. `MemPrintResize` computes a 64-bit FNV-1a hash over the callee's name, the name of the function that contains the call after inlining, and the call's debug location chain from the innermost location outward through every `inlinedAt` entry, each as file, line and column. An allocation inside an inlined

Callee	Runtime entry point	Kind
`malloc`	`__mpr_alloc(n, site, kind, 0)`	malloc
`calloc`	`__mpr_calloc(count, n, site)`	calloc
`realloc`	`__mpr_realloc(p, n, site)`	realloc
`aligned_alloc`, `memalign`, `valloc`	`__mpr_alloc(n, site, kind, align)`	memalign
`posix_memalign`	`__mpr_posix_memalign(out, align, n, site)`	memalign
`operator new`, `new[]`, aligned	`__mpr_alloc(n, site, kind, align)`	new
`operator new` (nothrow)	`__mpr_alloc(n, site, kind, 0)`	malloc
result of `vector<T>::_M_check_len`	`__mpr_capacity(len, 0, 1, site)`	(early pass)

Table 1: The calls `MemPrintResize` rewrites.

`std::allocator::allocate` therefore resolves to the program line that grew the container. The enclosing function's mangled name is part of the hash, because inlined library code has the same location for every element type. Without debug information the hash falls back to the function's name and the call's position in it, so the build needs `-g` (line tables are enough). The pass appends one line per site to a site map (`0x...  malloc main.cpp:777:5 ...`), which links a profile's sites to source lines. On darknet the pass rewrote 631 calls in 48 files, and the profiling and apply builds produced identical identifiers.

6.3 Vector growth capacity

A `std::vector` that grows by `push_back` or `emplace_back` doubles its capacity, copies its elements, and frees the old buffer. libstdc++ computes the new capacity in `_M_check_len`, called from `_M_realloc_insert` and `_M_default_append`. After inlining that call no longer exists, so the early instance of the pass runs at `PipelineStartEP`, before inlining, and wraps the integer result of every `_M_check_len` call:

```
%len = call i64 @_ZNKSt6vectorI...E12_M_check_lenEmPKc(ptr %this, i64 1, ptr @.str)
%cap = call i64 @__mpr_capacity(i64 %len, i64 0, i64 1, i64 <site>)   ; uses of %len now use %cap
```

The element size is not known at that point, so the pass passes 1 and the capacity model is expressed in elements. `libmpr` returns the larger of `len` and the site's predicted final size, so the first growth allocates the final capacity and the copy chain disappears. A wrong prediction cannot break the program, because the capacity only grows, and an over-large capacity is backed lazily. On miniVite the pass rewrote 73 allocation calls and 17 growth points. The pattern is specific to libstdc++ (GCC 8 headers here), and Section 10 discusses the consequence of naming the site before inlining.

6.4 Embedding reservations

In apply mode the late instance reads the reservation file and adds it to the module as a constant string, `__mpr_embedded_reservations`. Every translation unit carries a copy, so the global has `weak_odr` linkage in a COMDAT group of the same name, and the linker keeps one. (`weak_any` kept 48 copies in darknet's shared library, and `linkonce_odr` was discarded as unreferenced.) `libmpr` declares the symbol weak and reads it at start-up when `MPR_RESERVATIONS` is not set, so an apply build needs no environment.

6.5 Toolchain notes

clang parses `-mllvm` options before it loads a `-fpass-plugin`, so the plugin's options are not yet registered when they are parsed. The pass therefore reads `MPR_PASS_MODE`, `MPR_PASS_RESERVATIONS` and `MPR_PASS_SITEMAP` from the environment, and the same settings remain available as options under `opt`. On our machine clang found a newer GCC's headers first, so builds pass `-gcc-install-dir` for the system GCC 8, whose libstdc++ the programs run with. A build script (`scripts/resize_build.sh`) builds the plugin and `libmpr`, sets these variables, and runs the program's own build command. Two `opt`-plus-FileCheck tests check the rewriting of allocation calls and of a growth point, and an end-to-end test builds a small program in both modes.

7 The `libmpr` runtime

`libmpr` is about 760 lines of C. It defines `malloc`, `calloc`, `realloc`, `reallocarray`, `free`, the aligned allocators, `malloc_usable_size`, the `operator new` variants and `memset`, so it serves both a preloaded binary and a binary built with `MemPrintResize`. A block smaller than `MPR_MIN` (64 KB) or at a site without a reservation goes to glibc through `__libc_malloc` and its siblings, unchanged.

7.1 Serving a block

For a large block at a reserved site, `libmpr` computes the prediction p from the site's model, rounds the request n and p up to pages, and creates one anonymous mapping of the full size with `MAP_NORESERVE`. It then maps the first p bytes again with `MAP_FIXED` and without `MAP_NORESERVE`, which makes the prefix accountable. The program receives a pointer to all n bytes. If it touches beyond p, the kernel backs the page as for any anonymous memory, with no signal the program could observe. A design that protected the remainder with `PROT_NONE` and grew the block on a fault would also break system calls, because a `read()` into a protected page returns `EFAULT` instead of faulting. A `dead` site's block is entirely `MAP_NORESERVE`. A large `calloc` needs no `memset`, because fresh anonymous pages are zero. Blocks live in an open-addressing table keyed by address, and `free` recognises a runtime block by its page alignment and a table lookup, records its resident bytes with `mincore` for telemetry, and unmaps it. The tables are themselves mapped `MAP_NORESERVE`, so `libmpr` commits nothing of its own.

7.2 Growing and shrinking

A runtime block grows without copying where possible. `libmpr` first tries to map the new tail directly after the block with `MAP_FIXED_NOREPLACE`. If the address is taken, it reserves a new range and moves the block's two mappings into it with `mremap(MREMAP_FIXED)`, which moves page tables instead of data. Growth is geometric, at least $1.5\times$, so a table grown one element at a time does not remap at every step, and a shrink is kept in place unless it halves the block. We found two kernel behaviours on the way. The kernel keeps a tail that was added in place and written separately as its own mapping, and `mremap` fails with `EFAULT` on a range that spans several mappings. A failed `mremap(MREMAP_FIXED)` has also already unmapped its destination. Open MPI's start-up (in `libpciaccess`) grows a table one element at a time past the size threshold and crashed on the first version. `libmpr` now maps the destination again and copies such a piece.

7.3 Skipping the zeroing of untouched pages

Programs that clear a buffer they have just allocated make every page of it resident. `libmpr` defines `memset`. When the value is zero, the range lies in a runtime block, and the block's first page contains the start of the range, `libmpr` reads the range's entries from `/proc/self/pagemap` [9] and writes zeros only to pages that need them:

- a page that is neither present (bit 63) nor swapped (bit 62) has never been touched since the block was mapped, and it reads as zero, so it is skipped;
- a page that is present but not mapped exclusively (bit 56 clear) is the kernel's shared zero page, mapped by an earlier read, or a page shared with a forked child, so `libmpr` reads it and skips it if it is all zeros, which is correct in both cases;
- every other page, and the partial pages at both ends, are written.

Other `memset` calls go to glibc's implementation, found with `dlsym(RTLD_NEXT)` at start-up (a byte loop that GCC must not turn back into a `memset` serves the calls made before that).

The first version checked only the present bit. llama.cpp clears its KV cache a second time after a warm-up decode, and attention had read the whole cache by then, so every page mapped the zero page and looked present. After the zero-page check, 123 MB of the cache were still resident. The cause was transparent huge pages, which this machine enables for all anonymous memory: the warm-up writes a few rows into each layer's K and V tensors (about 3 MB each), and each first write faults in a 2 MB page [11]. `libmpr` therefore marks the pages whose zeroing it skipped `MADV_NOHUGEPAGE`. Those are the pages it chose to leave untouched, and a huge page would undo the choice. Pages the program has touched keep huge pages. `MPR_THP=0` and `MPR_THP=1` disable huge pages for all runtime blocks or leave them as the system sets them, and `MPR_ZERO=0` turns the zero skipping off.

7.4 Telemetry and settings

With `MPR_REPORT`, `libmpr` writes per site at exit the number of blocks, the bytes requested, predicted (committed), resident at free, the blocks that used more than predicted, and the zeroing it skipped. `__mpr_committed()` returns the predicted committed total for a scheduler to read. `MPR_SCALE` multiplies every prediction for robustness tests, and `MPR_ALL` serves every large block, reserved or not, to measure the mechanism alone. Two lock-related fixes were needed: `dladdr`, used to name a return-address site, is called outside `libmpr`'s lock, and the exit handler releases the lock before it writes the report, because `fclose` frees its buffer through `free`.

8 Evaluation

8.1 Setup

All runs used a two-socket Intel Xeon E5-2620 v4 (16 cores, 32 threads) with 62 GB of RAM, Linux 4.18 (RHEL 8) with heuristic overcommit and transparent huge pages set to `always`, GCC 8.5 and clang 21. Each configuration ran three times, and we report the mean of the peak RSS and the peak commit, sampled every 20 ms for the program and its children. Outputs were compared by hash: all printed lines except those that report a time or a rate, the modularity line for miniVite, the predictions for darknet, and the generated text for the LLM engines.

The workloads are:

	glibc		libmpr, own profile		libmpr, held out		Output
Workload	RSS	Commit	RSS	Commit	RSS	Commit	identical
darknet AlexNet	287.3	534.7	287.6	284.6	287.5	519.1	yes
miniVite 32768 (pass)	30.3	46.2	27.0	26.1	27.0	45.8	yes
llama.cpp 135M, 32 tok	379.9	404.5	193.2	265.1	191.0	265.1	yes
llama.cpp 135M, 128 tok	379.9	404.5	193.3	265.1	194.0	265.1	yes
llama.cpp 360M, 64 tok	768.1	560.5	435.4	419.9	436.1	425.6	yes
llama2.c 15M	66.9	6.3	67.1	6.2	67.1	4.1	yes
llama2.c 42M	201.2	36.2	197.6	32.8	199.3	34.0	yes
llama2.c 110M	498.3	78.2	486.8	72.8	496.1	69.9	yes

Table 2: Peak RSS and peak commit in MB (mean of 3 runs). miniVite's glibc column is the GCC build; its "own profile" column is the `MemPrintResize` build with the predicted `edgeList` capacity, and its held-out column is `libmpr` preloaded into the GCC build with reservations from the 8192 and 16384 inputs. All other rows are `libmpr` preloaded, except that darknet's apply build with embedded reservations gives the same 284.6 MB.

- darknet AlexNet inference on 10 images [17], built with GCC (preloaded) and with `MemPrintResize`;
- miniVite [4] with 32768 vertices and 4 OpenMP threads, built with GCC and with `MemPrintResize`;
- llama.cpp b11500 [3], CPU only, 4 threads, greedy decoding of "Once upon a time" with SmolLM2-135M-Instruct (32 and 128 tokens) and SmolLM2-360M-Instruct (64 tokens), both quantised to Q8_0 [1], at the model's default 8192-token context;
- llama2.c [5], greedy, 128 steps, with the stories15M, 42M and 110M models;
- the 23 programs of the scheduling corpus [14] at their largest inputs: PolyBench kernels, GAP, HPCCG, LULESH, miniFE, Kripke, Quicksilver, NPB, darknet, miniVite, the Lua, Perl and Python interpreters, SQLite and GCC's cc1.

For the LLM engines and the corpus, `libmpr` is preloaded, and each input has an own profile and a held-out profile from the program's other inputs (for darknet, the reference and Extraction networks).

8.2 Headline results

Table 2 and Figure 2 summarise the results. `libmpr` reduces peak commit by 46.8% on darknet, by 34.5% on llama.cpp with the 135M model and by 25.1% with the 360M model. It reduces peak RSS by 49.1% and 43.3% on llama.cpp and by 11.5% on miniVite through the pass's vector capacity, relative to the pass build without it. Every run of every workload produced glibc's output, and run time stayed within 5% of glibc for every LLM run (darknet 2%). The two rows where `libmpr` does little, llama2.c and darknet's held-out profile, are explained in Section 8.5 and Section 8.3.

8.3 darknet: dead training buffers

Figure 3 shows what the profile found. darknet's largest site allocates the `weight_updates` buffers of its connected layers (`connected_layer.c:35`) and convolutional layers (`convolutional_`

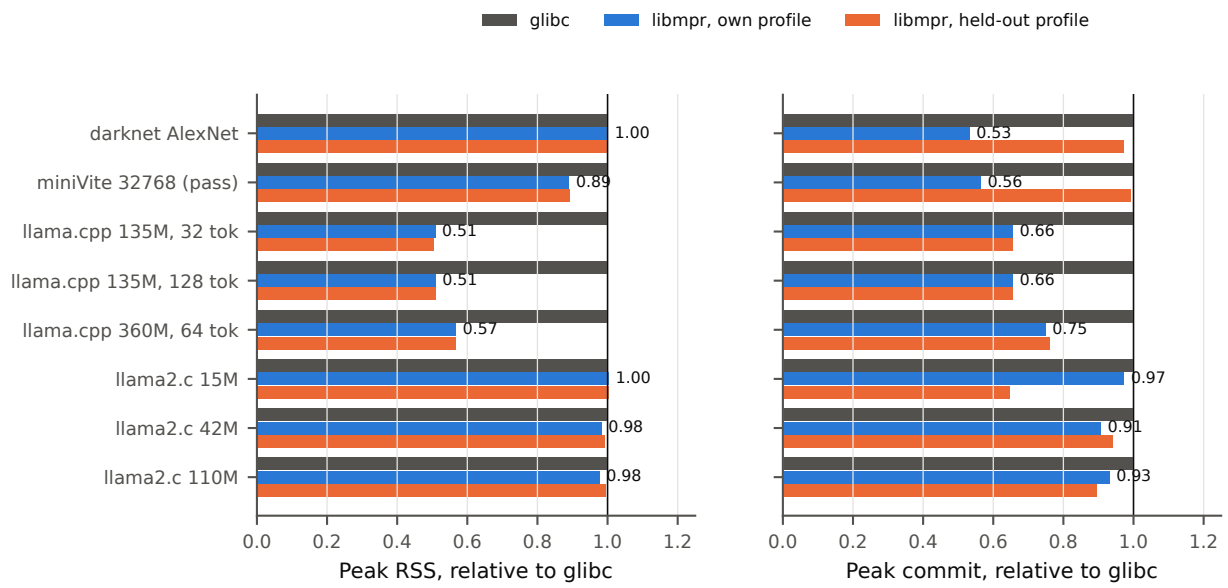


Figure 2: Peak RSS and peak commit under `libmpr`, relative to glibc. Every bar is a run whose output was identical to glibc's.

`layer.c:196`), which only training uses, so inference never touches them. With its own profile, `libmpr` maps them `MAP_NORESERVE`, and peak commit falls from 534.7 MB to 284.6 MB, close to the 270 MB the program touches. RSS does not change, because the untouched buffers were never resident. The `MemPrintResize` apply build, with the reservations embedded and no environment, gives the same 284.6 MB. A held-out profile from the reference and Extraction networks saves only 15.6 MB, because those networks have no fully connected layers: the profile has no data about the connected layers' site, which holds 234 MB of AlexNet's untouched buffers, and only the convolutional layers' 15 MB carry over. A profile helps the code paths it covers.

8.4 miniVite: vector growth and where its memory goes

miniVite touches 15.4 MB at its peak but holds 28.7 MB of RSS at its peak RSS and commits 44.0 MB at its peak commit (Figure 4, left). The breakdown, from `/proc/<pid>/smaps` at each peak, explains the gap:

- **Code, not data, is a third of the RSS.** 9.6 MB is library and program code (Open MPI, libstdc++, libgomp), which MemPrint does not count because it measures data references, not instruction fetches.
- **Vector growth.** `edgeList` (`graph.hpp:777`) grows by `emplace_back` without `reserve`, and it is the only large heap site live at the peak. When it doubles, the old 6.3 MB buffer and the new 12.6 MB buffer are both live (18.9 MB requested, 12.8 MB touched), and huge pages back the new buffer's 6.5 touched MB with about 8 MB.
- **Small blocks pay glibc's per-chunk overhead.** 1.8 million blocks smaller than 64 KB (hash table nodes of 24 to 47 bytes and small vectors) request 57 MB over the run but occupy 81 MB of glibc chunks (1.42×), and the heap holds them in 10.4 MB at the peak.
- **Untouched library memory.** Open MPI's start-up allocates 1.6 MB that it never touches.

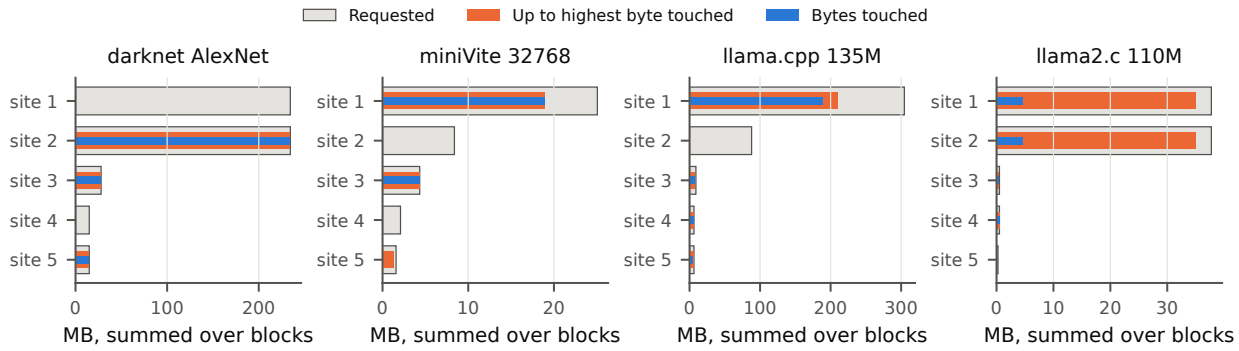


Figure 3: The five largest allocation sites of four workloads, summed over their blocks: bytes requested, bytes up to the highest touched offset (what a prefix reservation must commit), and bytes touched. darknet's first site is dead; llama2.c's two KV caches are touched sparsely across the whole block.

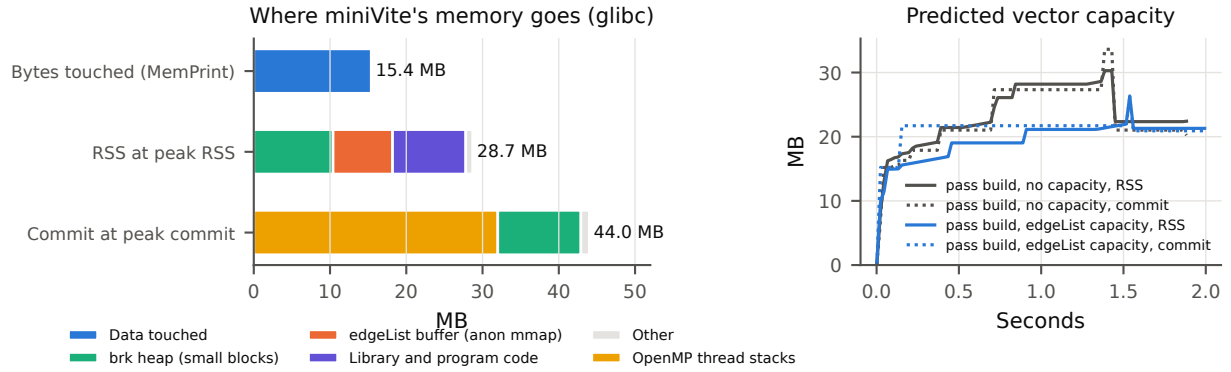


Figure 4: Left: miniVite's bytes touched, its RSS at peak RSS and its commit at peak commit under glibc, by mapping (smaps, 32768 vertices, 4 threads). Right: RSS and commit over time of the `MemPrintResize` build without and with the predicted `edgeList` capacity.

- **Commit is mostly thread stacks.** 32 MB of the peak commit is four 8 MB OpenMP thread stacks, of which 2 MB is resident. A smaller `OMP_STACKSIZE` would remove that commit, which no allocator change can.

The growth is what `MemPrintResize` can change. The profile of the 32768-vertex input finds that the growth site's final capacity is 271,116 elements (6.5 MB). With that capacity reserved in the pass build, the vector allocates once instead of eight times (25 MB in total), and peak RSS falls from 30.5 MB to 27.0 MB and peak commit from 33.6 MB to 26.1 MB, with the same modularity and run time (Figure 4, right). Preloaded into the GCC build with held-out reservations, `libmpr` reduces RSS from 30.3 MB to 27.0 MB through fresh zero pages for the growth buffers, but it cannot change the capacity.

8.5 LLM inference

llama.cpp. With the 135M model, llama.cpp allocates three large buffers through `ggml_aligned_malloc`: a 188.7 MB KV cache for 8192 tokens, a 115 MB compute buffer whose highest touched offset is 20.6 MB, and two 44 MB scheduler arrays (`ggml_backend_sched_new`) that are never touched. It clears the whole KV cache with `memset` at start-up and again after its warm-up decode. Under

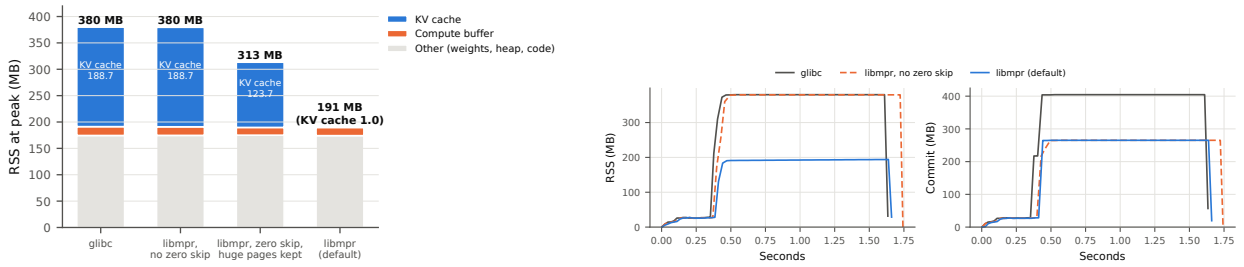


Figure 5: llama.cpp with SmolLM2-135M. Left: RSS at the peak by buffer, measured from `/proc/<pid>/pagemap` over each buffer's address range (32 tokens; "other" is the mapped weights, the heap and code). Right: RSS and commit over a 128-token run.

23 programs, largest inputs	glibc	libmpr, own	libmpr, held out
Total peak commit (MB)	2,504	2,249	2,487
Total peak RSS (MB)	2,293	2,285	2,286
Median commit / touched	1.01	1.01	1.01
Median RSS / touched	1.05	1.05	1.05
Time relative to glibc (range)	1	0.91 to 1.06	0.94 to 1.04
Programs with identical output	–	23 of 23	23 of 23

Table 3: The 23 programs of the scheduling corpus at their largest inputs (3 runs each).

glibc, the whole cache is therefore resident although a 32-token completion uses 0.4% of it. The model's weights are mapped from the GGUF file, so their 143 MB count in RSS but not in commit.

The two savings have different causes (Figure 5). Commit falls from 404.5 MB to 265.1 MB because of the reservations: the scheduler arrays are `dead` and become `MAP_NORESERVE`, and only the compute buffer's 20.6 MB prefix is committed. The KV cache stays committed in full, because the profile counted the zeroing as touching all of it. RSS falls from 379.9 MB to 193.2 MB because of the zero skipping. To attribute it, we recorded each buffer's address and measured its resident pages at the peak from `/proc/<pid>/pagemap` under four configurations. Without zero skipping, the KV cache holds 188.7 MB, as under glibc. Skipping the zeroing alone leaves 123.7 MB, all of it huge pages faulted in by the warm-up's writes of one row per layer. Skipping the zeroing and keeping huge pages off the skipped pages leaves 1.0 MB. The compute buffer holds 15 to 17 MB under every configuration, which is the part llama.cpp uses at the peak, and the remaining 174 MB (weights, heap and code) does not change. Held-out profiles from the program's other two inputs give the same results, because the sites are the same code paths at every model size. Run time does not change (0.85 s against 0.83 s for 32 tokens, 2.04 s against 2.14 s for the 360M model).

llama2.c. llama2.c is not darknet-like in the same way. Its KV caches are also sized for the full sequence (37.7 MB each for the 110M model, of which 4.7 MB is touched for 128 of 1024 positions), but it allocates them with `calloc` and does not clear them, so their untouched part was never resident. They are laid out as [layer][position][dimension], so every layer's first rows are touched and the highest touched offset lies at 93% of the block (Figure 3). A prefix reservation must cover it, and commit falls by only 7% (78.2 MB to 72.8 MB). Committing only the touched pages would need a reservation shaped like the use, for example a period and a prefix per period, which we have not built. RSS is dominated by the memory-mapped weights and does not change.

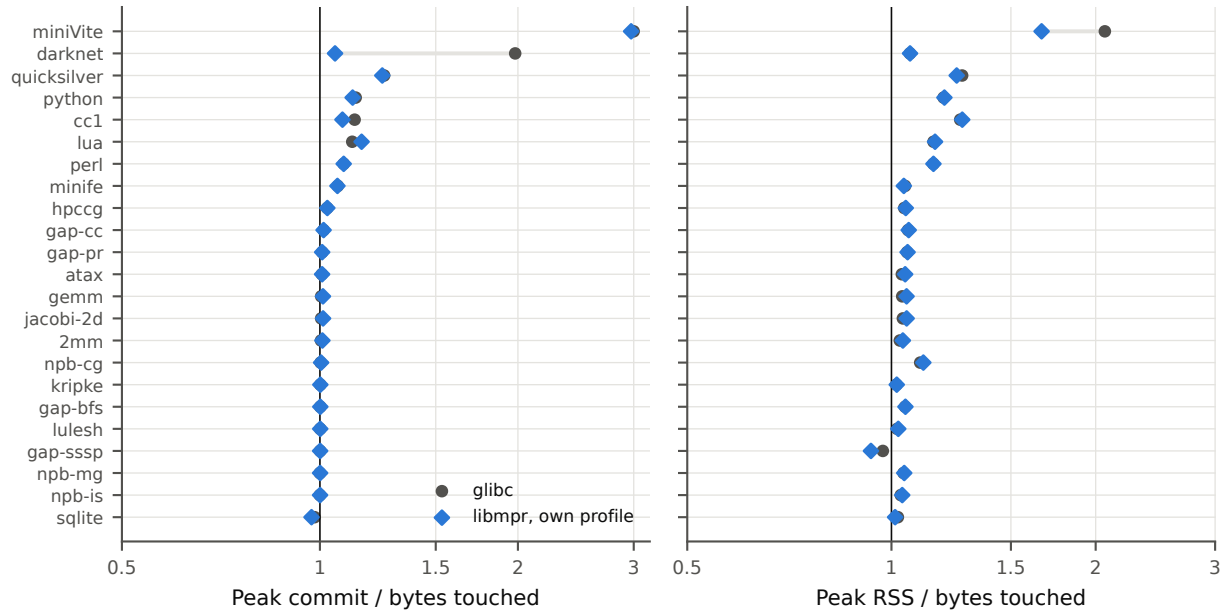


Figure 6: Peak commit and peak RSS relative to bytes touched for the 23 corpus programs, under glibc and under `libmpr` with each program's own profile. Most programs already sit at about 1.

8.6 The corpus: no regressions

Table 3 and Figure 6 show the 23-program corpus. Under glibc, the median program commits 1.01× its bytes touched and holds 1.05× in RSS, so there is little for a resizer to take. The exceptions are darknet (1.98× commit), miniVite (3.00×), Quicksilver (1.25×), cc1 and Python (1.13×) and Lua (1.12×). With own profiles, total peak commit falls by 10.2%, from 2,504 MB to 2,249 MB, nearly all of it darknet's. Held-out profiles save 0.7%. Lua's commit rises by about 2 MB, because its tables grow by `realloc` and leave the heap for runtime blocks. Every program produced glibc's output in every run, and run time stayed between 0.91× and 1.06× of glibc. The second version of `libmpr`, with zero skipping and the huge-page policy, changed no program's result relative to the first: total RSS moved from 2,286 MB to 2,285 MB.

8.7 Robustness to wrong predictions

Under-prediction is safe by construction, and we tested it. With every prediction multiplied by 0.1 (`MPR_SCALE=0.1`), darknet and miniVite produced identical outputs, and the telemetry reported the blocks that used more than predicted. The quantile in the fit leaves blocks uncovered by design, and held-out profiles under-predict on the LLM engines and the corpus, again with identical outputs. Over-prediction costs commit and nothing else, because the committed prefix is still backed lazily. Two control programs with no waste, PolyBench 2mm and HPCCG, with `libmpr` serving every large block (`MPR_ALL`), changed by at most 1.8 MB of RSS and 0.1 MB of commit.

8.8 Effect on scheduling

Our earlier study replayed the corpus's measured footprints through a simulator of 32 nodes with a shared memory pool [14]. We added a third truth, the RSS of each program under `libmpr` (Table 4). The results hardly move from the RSS truth, because the corpus programs' resident

Oracle	Truth: bytes touched	Truth: RSS	Truth: RSS under libmpr
Exact curve	90	91	103
Exact peak	460	436	457
MemPrint W5	391	1,807	1,727
DHAT peak	979	1,277	1,230
User request	2,215	2,197	2,204

Table 4: Mean response time (minutes) at 80% offered load in the DMSched-Sim scheduler of our earlier study [14], with calibrated margins, under three truths of what a job holds. The third truth is each program's RSS when it runs with `libmpr` and its own profile.

MemPrint curve	Median margin	90th percentile	Response (min)
Full trace, bytes, references	0.06	0.58	1,847
Full trace, bytes, seconds	0.08	0.44	1,476
Full trace, pages, references	0.03	0.56	1,942
Full trace, pages, seconds	0.05	0.41	1,423
W5, bytes, references	0.04	0.58	1,807
W5, bytes, seconds	0.04	0.56	1,820
W5, pages, references	0.04	0.58	1,979
W5, pages, seconds	0.04	0.49	1,705
Exact RSS peak	–	–	436
Exact RSS curve	–	–	91
User request	–	–	2,197

Table 5: MemPrint's footprint curves under the RSS truth, with the Pin tool's clock and unit options: the margin (headroom as a share of the curve's peak) each program needs for the curve never to fall below its RSS, over 23 programs, and the mean response time at 80% load with calibrated margins.

memory hardly moved: the exact curve gives 103 minutes (91 under the RSS truth) and MemPrint's windowed curves 1,727 (1,807). The scheduling study's conclusions therefore do not depend on resizing allocations, since its gain comes from reserving a curve instead of a peak. Resizing matters for programs that commit or hold much more than they touch, such as darknet and llama.cpp, and for systems with strict overcommit, where the saved commit is memory that can be promised to other jobs.

9 Discussion

9.1 The time axis of a footprint curve

In the scheduling study, a MemPrint curve needed about 60% headroom to cover the RSS truth, and we attributed part of that to the time axis, since MemPrint counts memory references while RSS is sampled in seconds [14]. The new options let us test the attribution. We reran the full trace and the 5% windowed mode at every program's largest input with `-footprint pages`, and mapped each curve to the wall time recorded in the `Seconds` column (Table 5, Figure 7). On wall time, the full trace's 90th-percentile margin falls from 0.58 to 0.44, and the mean response time from 1,847 to 1,476 minutes. The programs that gain most have phases that are short in references but long in time, such as GAP BFS (margin 0.53 to 0.06), GAP CC (0.56 to 0.09) and Lua (0.70 to 0.05). Counting pages instead of bytes helps the programs whose touched bytes are scattered (cc1 0.25 to 0.13, Perl 0.29 to 0.13, miniVite 1.12 to 0.84), but it does not reduce the calibrated

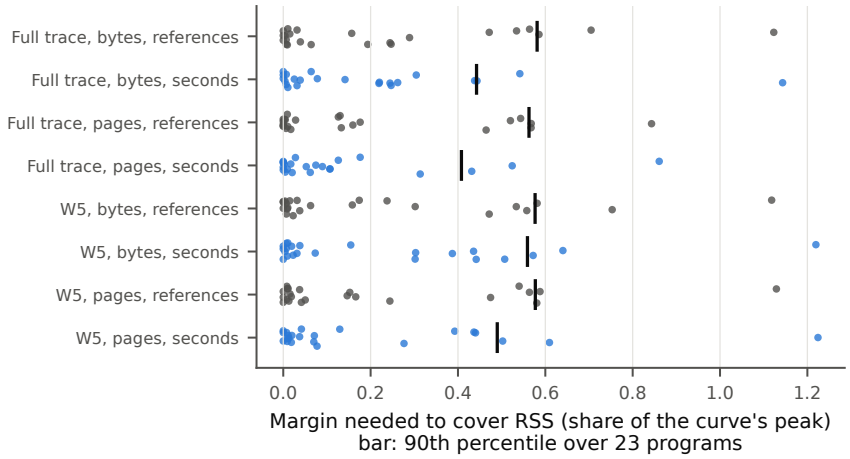


Figure 7: The margin each corpus program needs (dots; blue: wall-clock axis) and the 90th percentile over programs (bars), which sets the calibrated margin.

margin on its own. Together the two options give 0.41 and 1,423 minutes, still far from the exact peak's 436 minutes. The remaining margin comes from memory that MemPrint does not count, such as miniVite's library code, which the scheduler's RSS baseline (the run's smallest sample, taken before MPI loads its libraries) treats as footprint. The wall time is measured under Pin, which slows phases unevenly, so it approximates native time.

9.2 Bytes touched can exceed the resident set

Under `libmpr`, llama.cpp's RSS (193 MB) falls below its bytes touched as MemPrint measures them (353 MB at the peak, with the mapped weights). The difference is the zeroing `memset`: writing zeros to fresh anonymous memory counts as touching it, although no reader can distinguish the result from untouched memory. The Pin tool already records `calloc`'s own zeroing separately. Recording a program's zeroing of untouched memory in the same way would make the profile, and therefore the reservation, reflect what the program uses, and it would let `libmpr` commit less of the KV cache as well.

9.3 What the RSS numbers measure

The RSS figures in this paper are peaks of samples of `smaps_rollup`, taken every 20 ms, so a spike shorter than the interval can be missed. They include file-backed pages and count a huge page as 2 MB. The per-buffer figures for llama.cpp come from a different instrument. A logging library recorded where glibc (or `libmpr`) placed each buffer of at least 1 MB, without changing how it allocated, and at the sample with the largest total RSS we counted each buffer's pages that pagemap reports as present and mapped exclusively, which excludes the shared zero page. Under glibc the KV cache and the compute buffer lie in one merged mapping, so `smaps` alone cannot separate them.

9.4 Where resizing pays

The results separate three cases. A program that allocates buffers it never touches (darknet, llama.cpp's scheduler arrays) gains commit but no RSS, because the kernel never backed those

buffers. A program whose allocator or own code touches memory it does not need (a zeroing `memset`, vector doubling, huge pages over sparse writes) gains RSS, and only a change in how the memory is obtained recovers it. A program that already uses what it allocates, which is the median program in our corpus, gains nothing and loses nothing.

10 Limitations and future work

A limitation of prefix reservations is that a block is committed from its start up to the highest predicted offset, which commits most of a block that is used sparsely across its whole length, as llama2.c's KV caches are. A reservation shaped like the use, committing a prefix of each of several periods, would need the Pin tool to record the touched ranges of each block rather than only its highest offset.

A limitation of the vector capacity is that the early pass names the growth site before inlining, inside `_M_realloc_insert<T, Args...>`, so one capacity applies to every vector of that instantiation, and a capacity measured on one input does not transfer to another, because the final size depends on the input. Predicting it before the run would need MemPrint's static route, which estimates footprints from source and does not yet produce per-site profiles. The pattern is also specific to libstdc++.

Small blocks are left to glibc, so the per-chunk overhead of small allocations, 1.42× in miniVite, is not addressed. Allocators inside interpreters (pymalloc, Lua's tables) and custom `operator new` are not covered beyond the large blocks they request from the system.

The commit saving depends on the overcommit policy. Under strict overcommit the kernel ignores `MAP_NORESERVE`, so the saving disappears, although correctness is unaffected. The zero skipping reads pagemap, which unprivileged processes may read on this kernel without page frame numbers, and its zero-page check treats a page shared after `fork` conservatively by reading it.

Profiling is expensive: the Pin tool's exact mode with `-sites` took 328 s for a 0.86 s llama.cpp run and 528 s for llama2.c 110M (18 s natively), so profiles must be reused across runs, which the held-out results show is possible when the inputs exercise the same code paths.

11 Conclusion

We passed MemPrint's measured per-site footprints to a compiler pass and a runtime as non-binding reservations, so that a program is backed by what it touches rather than what it requests, and an under-prediction costs accounting instead of correctness. Every program we ran produced its original output. `libmpr` reduced darknet's commit by 46.8% and halved llama.cpp's resident memory, the pass's predicted vector capacity removed miniVite's growth chain, and the 23-program corpus, which already holds about what it touches, saw no regressions. The measurements also showed that a footprint curve is most useful to a scheduler on a wall-clock axis, and that the bytes a program touches include zeros it writes to fresh memory, which a resident set does not.

References

[1] Loubna Ben Allal et al. SmolLM2: When smol goes big – data-centric training of a small language model. arXiv:2502.02737, 2025.

[2] Jason Evans. A scalable concurrent malloc(3) implementation for FreeBSD. In *BSDCan*, 2006.

[3] Georgi Gerganov and ggml contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, release b11500, 2026.

[4] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[5] Andrej Karpathy. llama2.c: Inference Llama 2 in one file of pure C. `https://github.com/karpathy/llama2.c`, 2023.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[7] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2004.

[8] Huaicheng Li et al. Pond: CXL-based memory pooling systems for cloud platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[9] The Linux Kernel Documentation. *Examining Process Page Tables*. Documentation/admin-guide/mm/pagemap.rst.

[10] The Linux Kernel Documentation. *Overcommit Accounting*. Documentation/mm/overcommit-accounting.rst.

[11] The Linux Kernel Documentation. *Transparent Hugepage Support*. Documentation/admin-guide/mm/transhuge.rst.

[12] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[13] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. Learning-based memory allocation for C++ server workloads. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[14] Nafis Mustakin. What is an exact memory footprint worth to a scheduler? measured footprints and job scheduling on disaggregated memory. MemPrint repository, paper_sched, 2026.

[15] Nafis Mustakin, Joseph Manzano, and Daniel Wong. MemPrint: Constructing program memory footprint estimations using statistical methods from sparsely sampled Pin-based memory traces. Manuscript, 2025.

[16] Bobby Powers, David Tench, Emery D. Berger, and Andrew McGregor. Mesh: Compacting memory management for C/C++ applications. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2019.

[17] Joseph Redmon. Darknet: Open source neural networks in C. `https://pjreddie.com/darknet/`, 2016.

[18] Joe Savage and Timothy M. Jones. HALO: Post-link heap-layout optimisation. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization (CGO)*, 2020.

[19] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Memory demands in disaggregated HPC: How accurate do we need to be? In *International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2021.